Computer Organization, Spring 2026

# Project Assignment 1:

# Multiplier and Divider Design


Lecturer: [redacted]

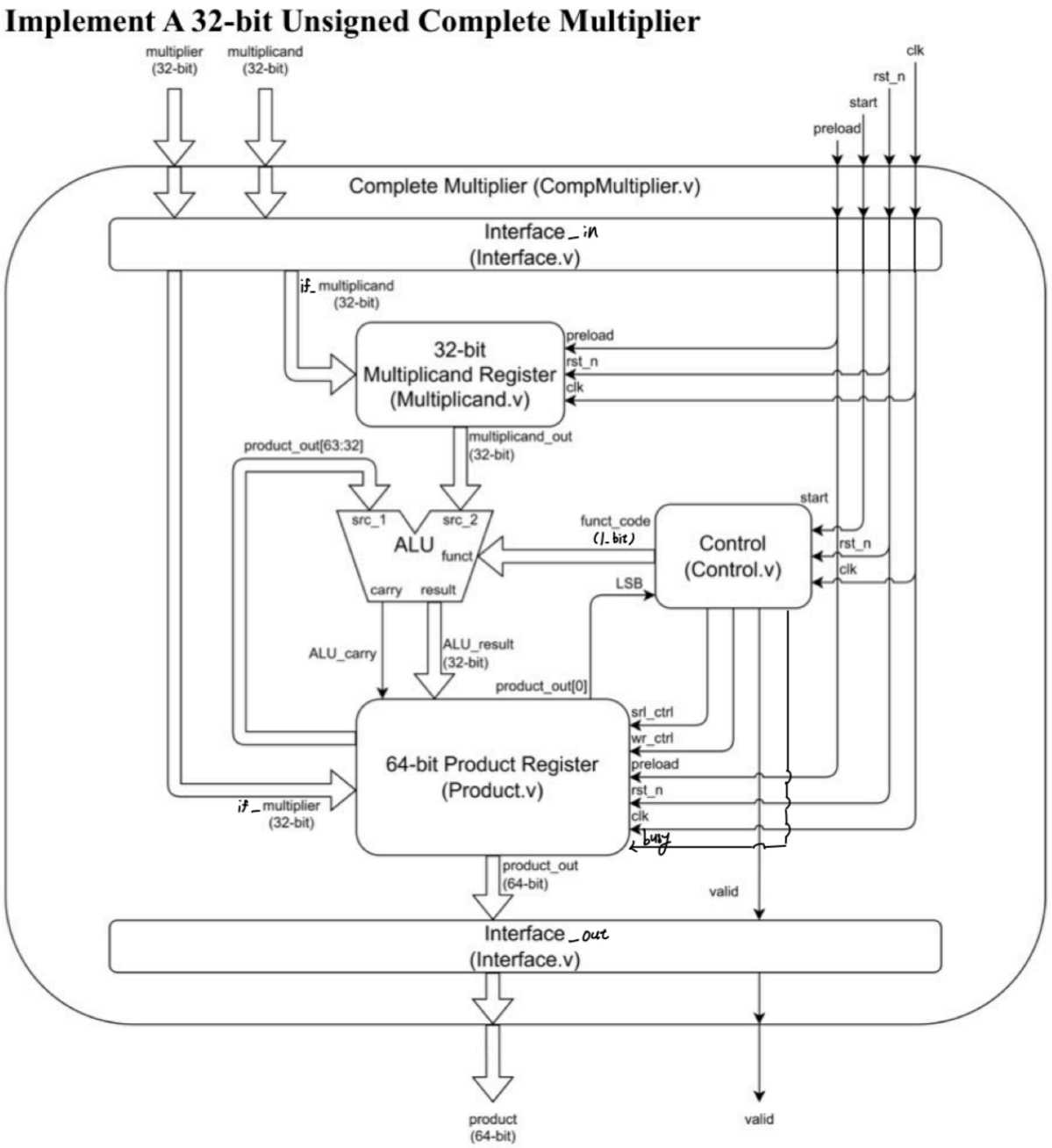
Department: Electrical Engineering

Student ID: [redacted]

Name: 胡守曾

# CompMultiplier.v

## Hardware Architecture




## Design Process

Interface.v,Multiplicand.v:

Interface負責接收外部傳入的資料給內部暫存器以及在運算結束後做輸出介面。而Multiplicand則是負責在preload階段把被乘數存起來，給ALU做後續運算使用。

ALU_Booth.v:

整個ALU功能只有加法，透過傳入funct來控制是否做加法。

Control.v, Product.v:

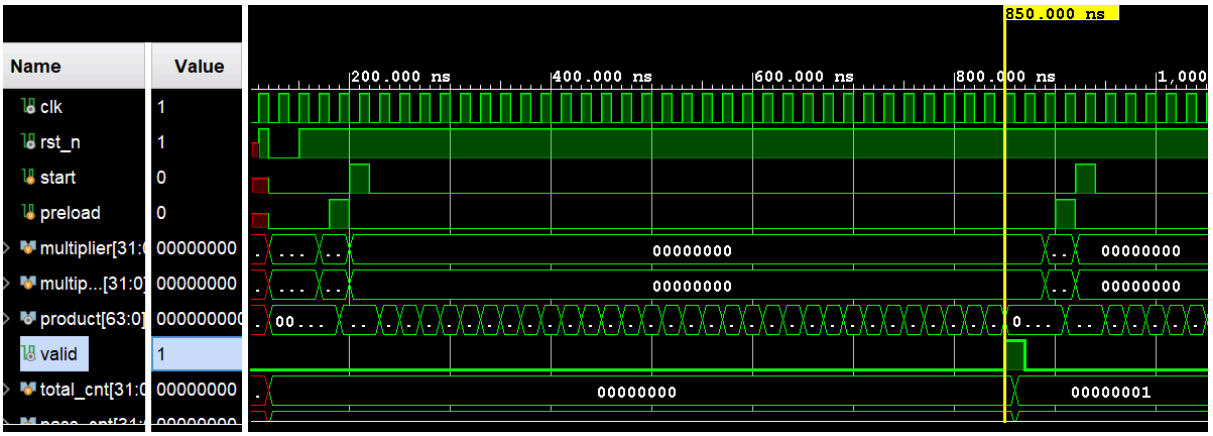
```
if (preload) begin
    product_reg <= {32'd0, multiplier};
end
else if (busy) begin
    temp_reg = {1'b0, product_reg};

    if (wr_ctrl) begin
        temp_reg[64:32] = {alu_carry, alu_result};
    end

    if (srl_ctrl) begin
        temp_reg = temp_reg >> 1;
    end

    product_reg <= temp_reg[63:0];
end
```

在Product中，為了在一個clk period 內同時完成 ALU 加法運算與位移動作，也就是不能同一週期內進行兩次連續更新，以及需要多1 bit來給alu_carry，我設立了一個65bits 的暫存器temp_reg。並且在接收Control 給的訊號的 wr_ctrl 與 srl_ctrl 訊號，決定 temp_reg 執行資料更新或右移，並將結果存至 product_reg。

```
assign wr_ctrl  = busy & lsb;
assign srl_ctrl = busy;
assign funct    = busy & lsb;
```

Control 為主要控制循環機制，首先會接收lsb (product_out[0])，並以此輸出wr_ctrl, srl_ctrl, 以及funct。首先wr_ctrl 為控制alu_result, alu_carry是否需要加入product_out。srl_ctrl 則是向右移的控制，雖然每次循環都會要向右移1bit，但還是在Control 內設定srl_ctrl 控制，因此其只要busy 為高電位則會一樣為高電位。以上兩個訊號都會輸出到Product，而funct則會輸出到ALU中，以此控制(Multiplier[0] = 1)時需要執行加法。

## Behavioral Simulation




```
run all
[PASS] 2026 * 2026 = 4104676
[PASS] 2863311530 * 1431655765 = 4099276458915470450
[PASS] 8230 * 4115 = 33866450
[PASS] 0 * 8230 = 0
-------------------------------------------------------
[SUMMARY] Total: 5, Pass: 5, Fail: 0
[RESULT] PASS: All tests passed.
-------------------------------------------------------
$finish called at time : 3770 ns : File "C:/Vivados_Ve
```

透過波形圖可以看到整個運算周期約是32個clk cycle，當product_out 完成運算後，valid 會被拉成高電位。在Tcl Console 顯示也可以看到輸出結果為正確。

## Synthesis Result

Timing:

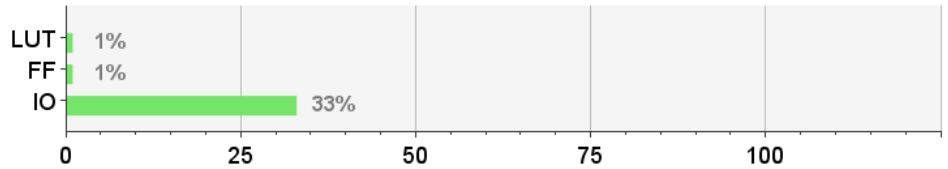
| Setup | | Hold | | Pulse Width | |
|---|---|---|---|---|---|
| Worst Negative Slack (WNS): | 0.991 ns | Worst Hold Slack (WHS): | 0.139 ns | Worst Pulse Width Slack (WPWS): | 7.000 ns |
| Total Negative Slack (TNS): | 0.000 ns | Total Hold Slack (THS): | 0.000 ns | Total Pulse Width Negative Slack (TPWS): | 0.000 ns |
| Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 |
| Total Number of Endpoints: | 375 | Total Number of Endpoints: | 375 | Total Number of Endpoints: | 105 |

**All user specified timing constraints are met.**

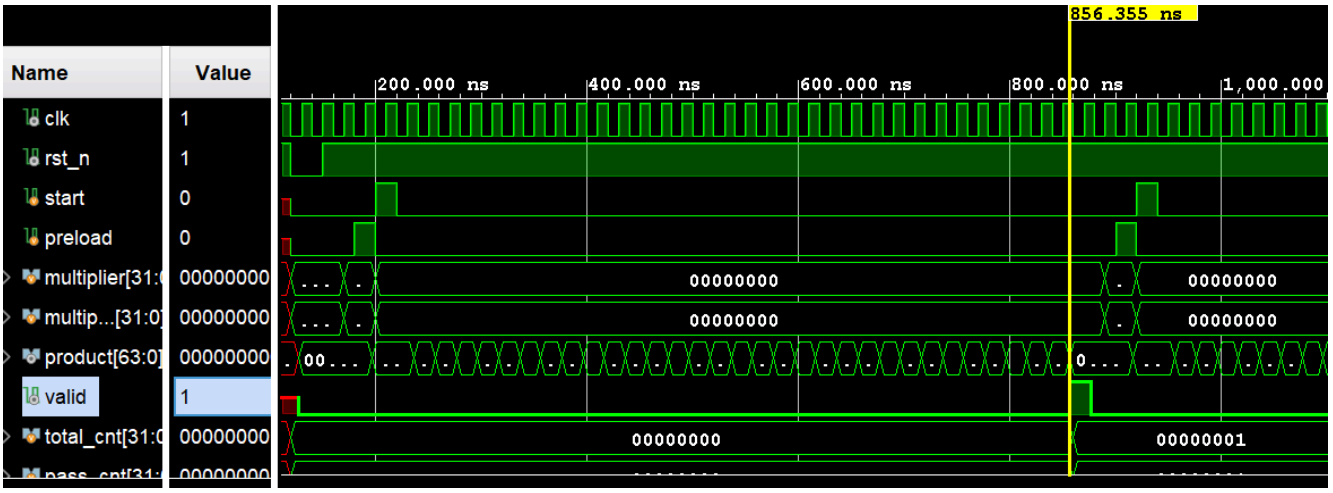
這次clk period設定為15 ns，可以看到WNS、WHS均為正值，失敗點也都是0，代表可以在當前clk頻率下運定運行，具有足夠Slack。

Hardware Utilization:

| Resource | Utilization | Available | Utilization % |
|---|---|---|---|
| LUT | 91 | 134600 | 0.07 |
| FF | 104 | 269200 | 0.04 |
| IO | 133 | 400 | 33.25 |




## Post_Synthesis Timing Simulation

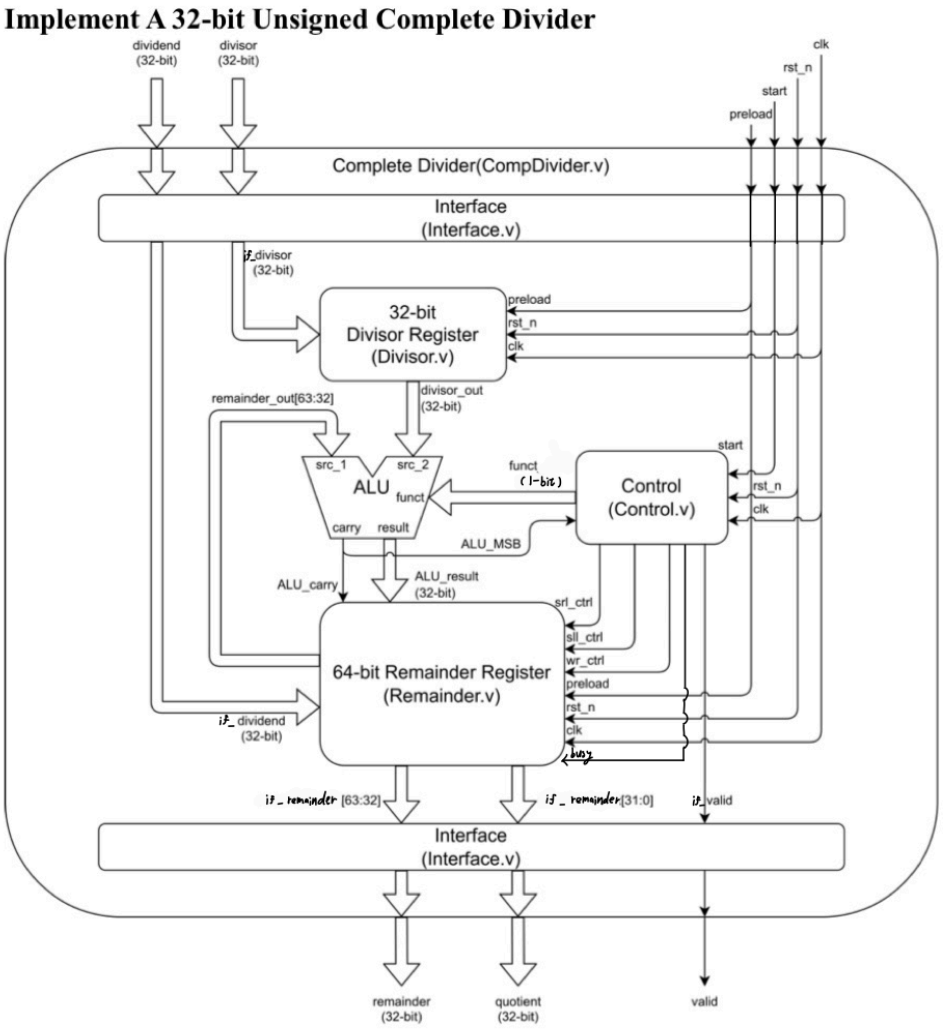



```
run all
[PASS] 2026 * 2026 = 4104676
[PASS] 2863311530 * 1431655765 = 4099276458915470450
[PASS] 8230 * 4115 = 33866450
[PASS] 0 * 8230 = 0
-----------------------------------------------------
[SUMMARY] Total: 5, Pass: 5, Fail: 0
[RESULT] PASS: All tests passed.
-----------------------------------------------------
$finish called at time : 3770 ns : File "C:/Vivados_V
```

透過波形圖一樣可以看到valid、product的輸出不會剛好在clk正緣處，反映出了在實際經過組合邏輯與佈線後的傳遞延遲。

# CompDivider.v

## Hardware Architecture



## Design Process

Interface.v, Divisor.v:

Interface負責接收外部傳入的資料並在運算結束後輸出。Divisor負責儲存32 bit除數，當preload訊號觸發時，將除數存至暫存器。

ALU_Div.v:

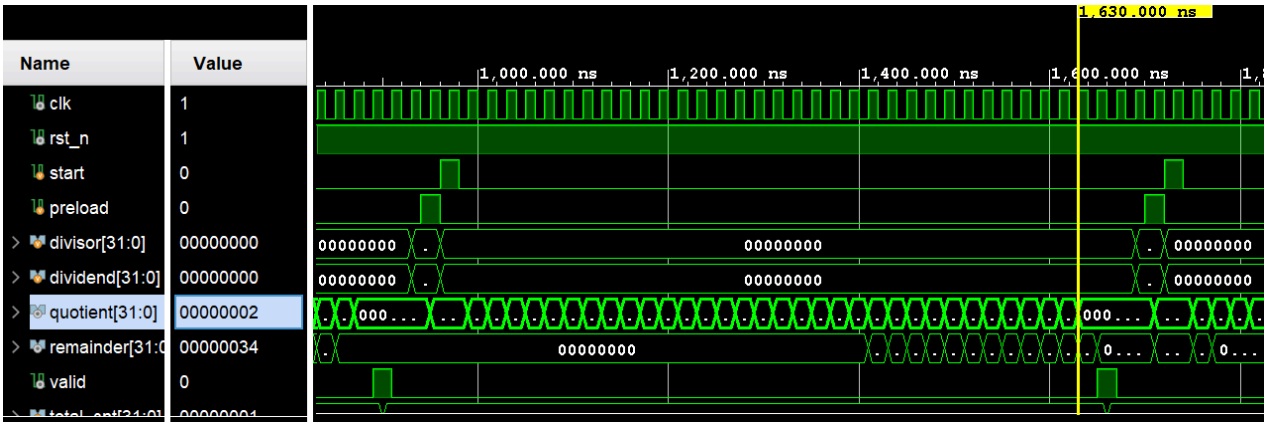
我去掉了加法，因為在Remainder裡面不會用到，僅設計成使用減法功能。

Control.v, Remainder.v:

```
else if(busy && wr_ctrl) begin
    if(sll_ctrl && srl_ctrl) begin
        remainder_reg <= {remainder_reg[62:0], 1'b0};  // initially left shift
    end
    else if(sll_ctrl) begin
        if(carry) begin  // negative
           remainder_reg <= {remainder_reg[62:0], 1'b0};  // not use alu_result
        end
        else begin  // positive
            remainder_reg <= {result[30:0], remainder_reg[31:0], 1'b1};
        end
    end
    else if (srl_ctrl) begin  // remainder right shift
        remainder_reg[63:32] <= {1'b0, remainder_reg[63:33]};
    end
```

為核心控制，透過內部計數器(count)管理32次運算。wr_ctrl 控制寫入訊號，僅在高訊號為高電位時允許Remainder暫存器更新。在運算起始階段，若 sll_ctrl 與 srl_ctrl 同時觸發，將執行初始化左移。進入正式運算循環後，由 sll_ctrl 驅動左移與減法：此時根據 ALU 輸出的 alu_carry（借位訊號）判定減法結果之正負。若結果為負，暫存器將忽略 alu_result 並維持原值左移，末位補 0；若結果為正（alu_carry 為 0），則將減法後的 alu_result 寫入暫存器高位，同步完成左移並於末位補 1。完成循環後，由srl_ctrl控制 remainder[63:32]右移。

## Behavioral Simulation




```
run all
[PASS] 2026 / 1000 = Q:2 R:26
[PASS] 4294967295 / 2 = Q:2147483647 R:1
[PASS] 305419896 / 4096 = Q:74565 R:1656
[PASS] 2147483648 / 3 = Q:715827882 R:2
--------------------------------------------
[SUMMARY] Total: 5, Pass: 5, Fail: 0
[RESULT] PASS: All tests passed.
--------------------------------------------
$finish called at time : 3970 ns : File "C:/
```

原本我設定valid會在執行完32次count計算後變拉成高電位，但是發現remainder還需要一個clk cycle(右移)才能達到正確的數值，因此我將valid調慢了一個clk升起，因此總共計算時間約是33個clk cycle。

## Synthesis Result

Timing:

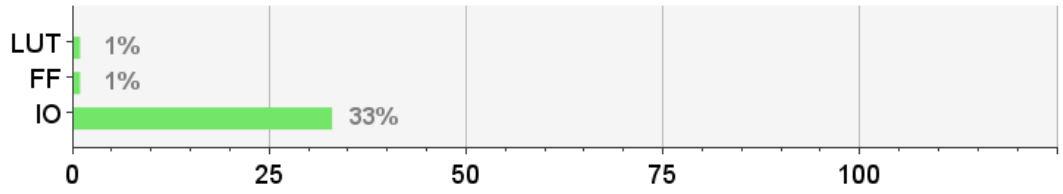
| Setup | | Hold | | Pulse Width | |
|---|---|---|---|---|---|
| Worst Negative Slack (WNS): | 3.491 ns | Worst Hold Slack (WHS): | 0.139 ns | Worst Pulse Width Slack (WPWS): | 9.500 ns |
| Total Negative Slack (TNS): | 0.000 ns | Total Hold Slack (THS): | 0.000 ns | Total Pulse Width Negative Slack (TPWS): | 0.000 ns |
| Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 |
| Total Number of Endpoints: | 381 | Total Number of Endpoints: | 381 | Total Number of Endpoints: | 108 |

**All user specified timing constraints are met.**

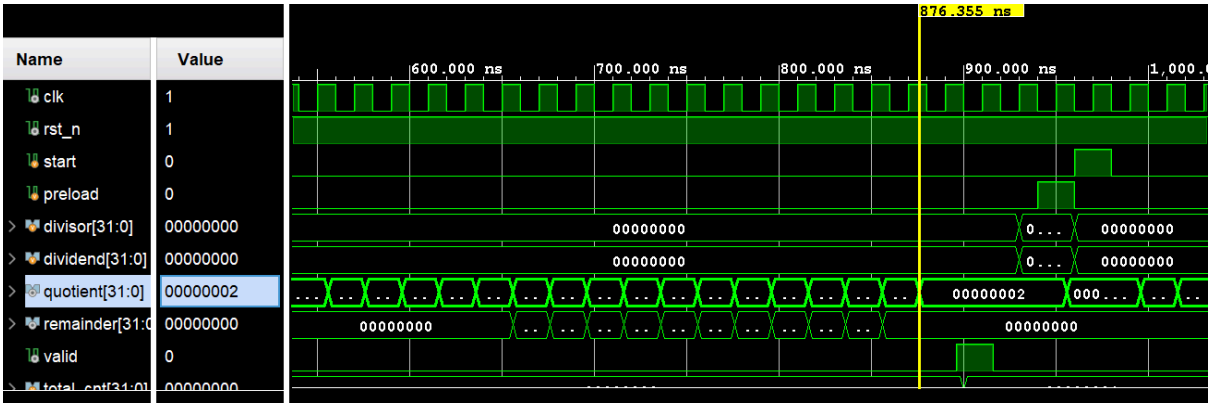
此次設計clk period 為20ns，WNS、WHS均為正值，失敗點也都是0，代表可以在當前clk頻率下運定運行，具有足夠Slack。

Hardware Utilization:

| Resource | Utilization | Available | Utilization % |
|---|---|---|---|
| LUT | 109 | 134600 | 0.08 |
| FF | 107 | 269200 | 0.04 |
| IO | 133 | 400 | 33.25 |




## Post_Synthesis Timing Simulation

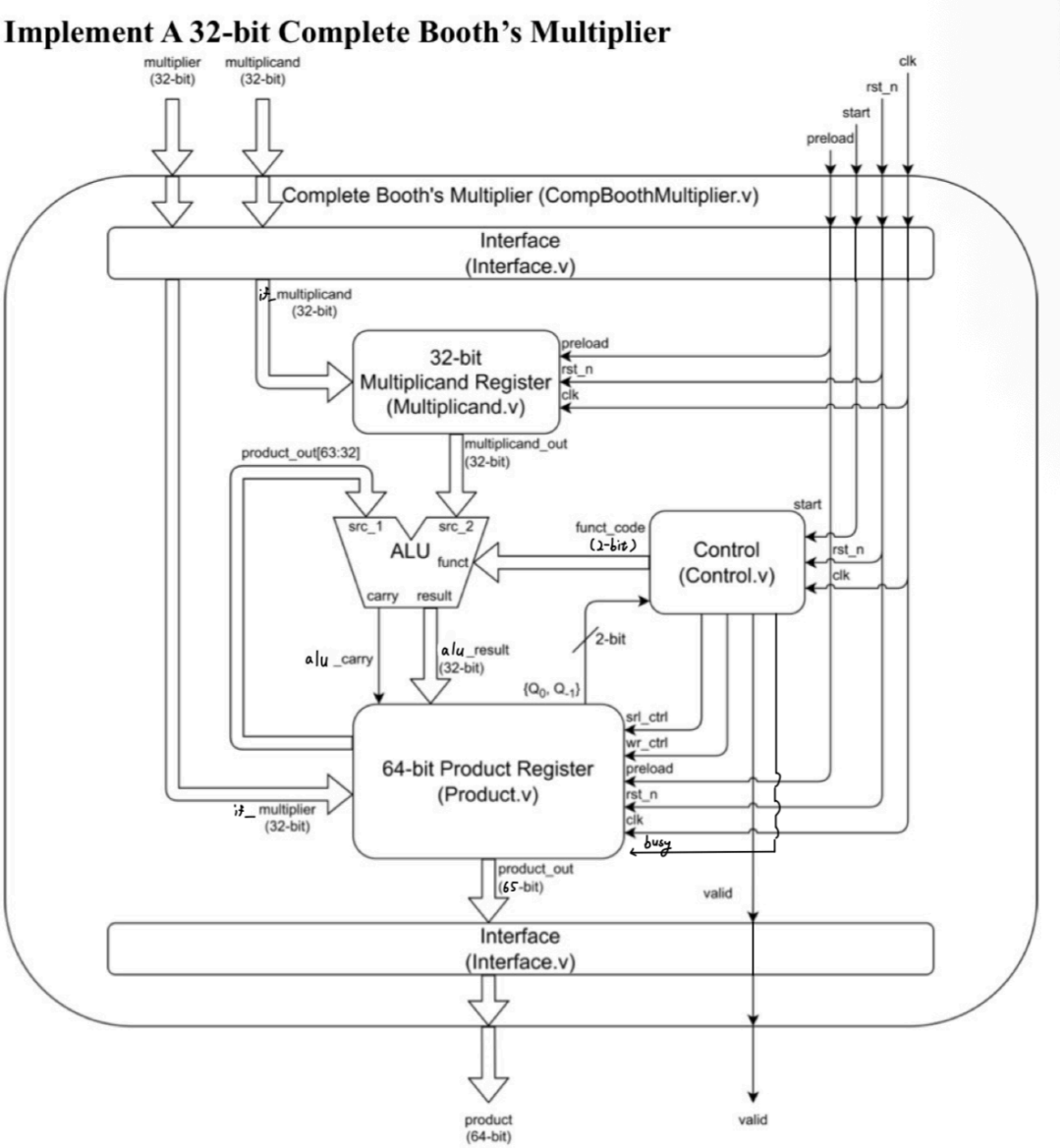



```
run all
[PASS] 2026 / 1000 = Q:2 R:26
[PASS] 4294967295 / 2 = Q:2147483647 R:1
[PASS] 305419896 / 4096 = Q:74565 R:1656
[PASS] 2147483648 / 3 = Q:715827882 R:2
-----------------------------------------
[SUMMARY] Total: 5, Pass: 5, Fail: 0
[RESULT] PASS: All tests passed.
-----------------------------------------
$finish called at time : 3970 ns : File "C
```

可以看到在quotient、remainder以及valid完成時都不是剛好在clk正緣處，這便是做好post_synthesis 後的結果，會有一段運算時間，因此clk period不能調整太低，不然會有錯誤。

# CompBoothMultiplier.v

## Hardware Architecture




## Design Process

Interface_Booth.v,Multiplicand_Booth.v:

Interface負責接收外部傳入的資料給內部暫存器以及在運算結束後做輸出介面。而Multiplicand則是負責在preload階段把被乘數存起來，給ALU做後續運算使用。

ALU_Booth.v:

這個ALU模組有加減法功能，為了應對不同的q0 以及q_1，一樣只是combinational Logic Circuit。因此不需要加上clk。

Control_Booth.v, Product_Booth.v:

```
assign funct = (busy) ? (
                  ({q0, q_1} == 2'b01) ? 2'b01 :   // funct = 01
                  ({q0, q_1} == 2'b10) ? 2'b10 :   // funct = 10
                                         2'b00     // default
                ) : 2'b00;                         // busy = 0, default
```

整個CompBoothMultiplier 的控制邏輯，主要由Control 模組主要負責判斷目前是否處於 busy 運算階段。一旦是便會去檢查product_out暫存器的最後兩位(也就是q0 以及 q_1)，透過比較這兩位的值，來決定這一輪要執行加法還是減法，並會在ALU內進行運算並輸出，並在每一輪運算後，都必須執行一次右移，因此沒有把執行右移的指令放在Control中。

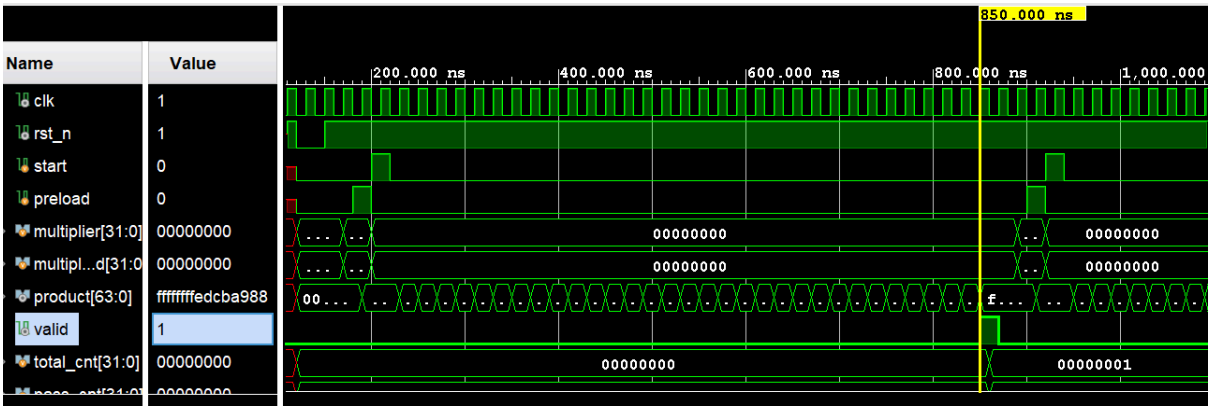
```
else if (busy) begin
    temp_reg = product_reg;

    case (funct)
        2'b01: temp_reg[64:33] = alu_result;   // A = A + M
        2'b10: temp_reg[64:33] = alu_result;   // A = A - M
        default: temp_reg[64:33] = product_reg[64:33];
    endcase
        temp_reg = {temp_reg[64], temp_reg[64:1]};  // right shift keep the first letter

    product_reg <= temp_reg;
```

temp_reg 以及 product_reg都是設計成65bits，第0位是存放q_1，在preload階段會先預設為0，接下來做的右移會改變它。

```
if (preload) begin
    count <= 6'd0;
    busy  <= 1'b0;
end
else if (start && !busy) begin
    count <= 6'd0;
    busy  <= 1'b1;
end
else if (busy) begin
    count <= count + 6'd1;

    if (count == 6'd31) begin
        busy  <= 1'b0;
        valid <= 1'b1;
    end
end
```

此為Control 在判斷循環的部分，設定一暫存器 count 來判斷是否達到總共32次循環。

## Behavioral Simulation




```
run all
[PASS] -2147483648 * 2 = -4294967296
[PASS] 2147483647 * 2 = 4294967294
----------------------------------------
[SUMMARY] Total: 3, Pass: 3, Fail: 0
[RESULT] PASS: All tests passed.
----------------------------------------
$finish called at time : 2330 ns : File
```

透過波形圖可以知道在preload階段Product就已經存入multiplier，並且當count達到31次並完成最後一次移位後，valid被拉高，此時輸出的Product便會是正確的值。

## Synthesis Result

Timing:

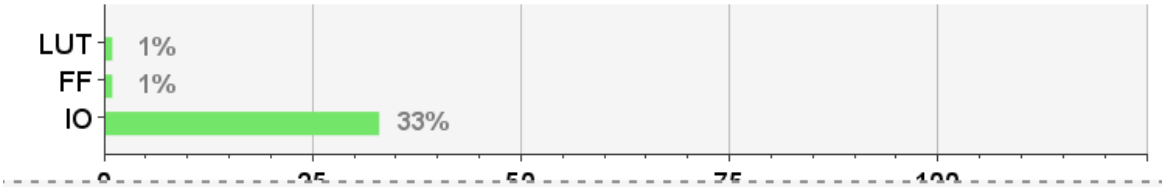
| Setup | | Hold | | Pulse Width | |
|---|---|---|---|---|---|
| Worst Negative Slack (WNS): | 0.991 ns | Worst Hold Slack (WHS): | 0.139 ns | Worst Pulse Width Slack (WPWS): | 7.000 ns |
| Total Negative Slack (TNS): | 0.000 ns | Total Hold Slack (THS): | 0.000 ns | Total Pulse Width Negative Slack (TPWS): | 0.000 ns |
| Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 |
| Total Number of Endpoints: | 377 | Total Number of Endpoints: | 377 | Total Number of Endpoints: | 106 |

**All user specified timing constraints are met.**

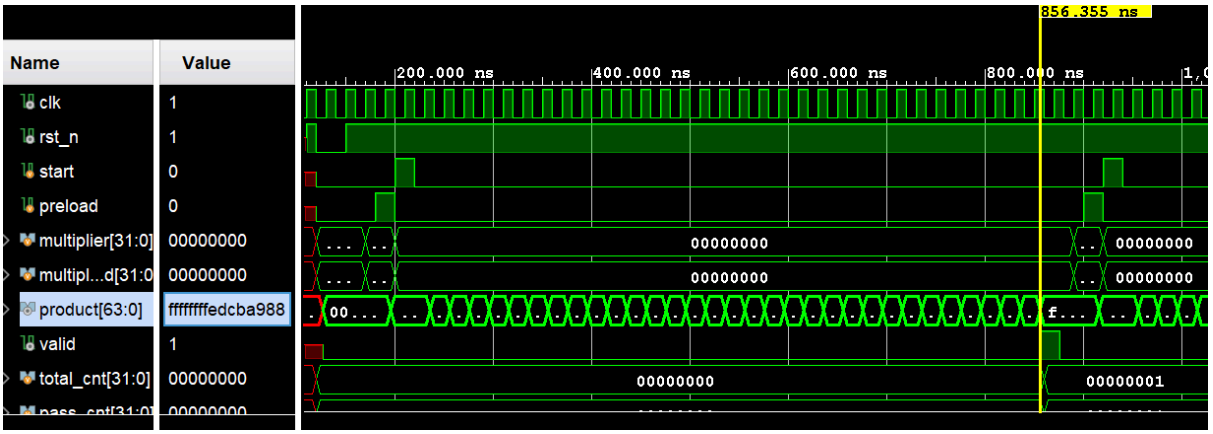
此次設計clk period 為15ns，WNS、WHS均為正值，失敗點也都是0，代表可以在當前clk頻率下運定運行，具有足夠Slack。

Hardware Utilization:

| Resource | Utilization | Available | Utilization % |
|---|---|---|---|
| LUT | 127 | 134600 | 0.09 |
| FF | 105 | 269200 | 0.04 |
| IO | 133 | 400 | 33.25 |




## Post_Synthesis Timing Simulation




```
run all
[PASS] -2147483648 * 2 = -4294967296
[PASS] 2147483647 * 2 = 4294967294
----------------------------------------
[SUMMARY] Total: 3, Pass: 3, Fail: 0
[RESULT] PASS: All tests passed.
----------------------------------------
$finish called at time : 2330 ns : File
```

透過波形圖一樣可以看到valid、product的輸出不會剛好在clk正緣處，反映出了在實際經過組合邏輯與佈線後的傳遞延遲。

# Conclusion

在這次報告總共完成了三種算術邏輯單元的設計，分別為Unsigned Multiplier、Unsigned Divider 以及 Booth's Multiplier。整個設計沒有使用Verilog 內建乘除算式，而是透過位移與加減邏輯達成。

整個IO Utilization使用率達到33%，高於 LUT 與 FF 的使用率。這是因為 32-bit 乘法器需要大量的輸入輸出腳位：包含乘數與被乘數各 32 位元、輸出 64 位元，再加上clk、rst_n與控制訊號，總計約 133 個腳位。

### CompMultiplier.v

透過 Control 判斷乘數的最低位元（lsb），決定是否觸發 ALU 執行加法。為了在單一週期內處理進位與右移，使用了 65-bit 的 temp_reg 阻塞賦值邏輯，避免了非阻塞賦值導致的時序競爭，確保 32 輪運算後獲得正確積值。

### CompDivider.v

使用恢復餘數的方法。透過 ALU 的借位（alu_carry）判斷試減結果：若不夠減則維持原值左移補 0（不寫入減法結果以達成恢復效果）；若夠減則寫入差值並左移補 1。最終透過 srl_ctrl 訊號修正餘數位置，輸出商數與餘數。

### CompBoothMultiplier.v

相較於無號乘法器，透過檢查{q0, q_1}}來簡化運算，使有號數(Signed)可以計算。整個重點在於如何使Control的輸出為正確的值，透過保留最高位元(Sign Extension)確保正負號在移位過程中不消失。

## 心得

這次的報告我完整實作了乘法器與除法器的迭代架構，使我對計算機組織正課的教學有更值觀的認識，也深刻體會到「硬體思維」與「軟體編程」的差異。在實作過程中，最大的挑戰在於時序控制與暫存器賦值的處理，特別是在處理 Booth 演算法的算術右移以及除法器的恢復餘數邏輯時，必須精確計算每一個時鐘週期的資料流向，若一不小心調整出錯便會導致結果有錯。

起初我將所有邏輯集中在同一模組中，然而隨著開發深入，我體會到模組化設計的重要性。將電路拆解為 Interface、ALU、Control 與各類 Register 後，不僅大幅提升了程式碼的可讀性，也讓 Debug 過程更具效率，我能針對特定模組進行獨立驗證，確保資料路徑與控制訊號正確對齊。

Original CompMultiplier.v

```
reg [31:0] multiplicand_reg;
reg [63:0] sum_reg;
reg [64:0] temp_reg;
//reg [63:0] product_reg;
reg [5:0]  count;
reg valid_reg;
reg busy_reg;

assign product = sum_reg;
assign valid = valid_reg;

always@(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        multiplicand_reg <= 32'd0;
        sum_reg          <= 64'd0;
        count            <= 6'd0;
        busy_reg         <= 1'd0;
        valid_reg        <= 1'd0;
        //product_reg      <= 64'd0;
    end
    else begin
        valid_reg <= 1'b0;
        if(preload)begin
            multiplicand_reg <= multiplicand;
            sum_reg <= {32'd0, multiplier};
            count <= 6'd0;
            busy_reg <= 1'd0;
        end
        else if(start && !busy_reg)begin
            count <= 1'd0;
            busy_reg <= 1'd1;
        end
        else if(busy_reg)begin
            temp_reg = {1'b0, sum_reg};
            if(sum_reg[0])begin
                temp_reg[64:32] = {1'b0, sum_reg[63:32]} + {1'b0, multiplicand_reg};
            end
            temp_reg = temp_reg >> 1;
            sum_reg <= temp_reg[63:0];
            count <= count + 1'd1;
            if(count == 6'd31)begin
                valid_reg <= 1'd1;
                busy_reg <= 1'd0;
                //product_reg <= temp_reg[63:0];
            end
        end
```